Digitisation - UI Discovery - High Level Design Overview

⚠ This page is a living document

Status	IN PROGRESS
Impact	HIGH
Driver	@james.wright1@greenflag.com
Approver	@john.mcdonnell@greenflag.com
Decision Contributors	@franco.parente@greenflag.com @manu.srinivasa@greenflag.com
Informed	
Due date	9 Jul 2025

High level requirements
Languages and Frameworks
Front End Architecture Proposal - ideal state
NX package based monorepo - front end libraries
Digitisation applications
Green Flag vs Direct Line Group infrastructure
CMS Integration
Testing approach
Observability (FE)
Observability (BFF)
Static code analysis
CSS Frameworks / Component Library
Tech Stack - specific tools

This document serves to investigate the technology, tools and high level design for the web build for Digitisation. It should be reviewed by the Development teams to validate any technical choices suggested and/or to propose alternative approaches if they are believed more appropriate. This is particularly appropriate for those the React

ecosystem tools, since existing journeys have historically been Angular based.

High level requirements

At a high level, digitisation involves a greenfield rebuild of:

- Greenflag brochureware site (currently written with Gulp and Panini and hosted on DLG Bitbucket instance whose devops responsibility lies with the Digital Dev Ops team)
- Green Flag blog, currently a Wordpress solution
- The existing direct Quote and Buy application, written with Angular 11

As well as rebuilding these existing customer facing elements, Digitisation also aims to add the following functionality:

- Consolidation of Capita hosted journeys (e.g. Business breakdown cover, PCW) into the new greenfield quote and buy application;
- Integration with the Group level chosen CMS (Contentful) to allow a subset of change to be delivered without any, or minimal engineering effort;
- Creation of customer facing ‘My Account’ area so that customers can view their policy details, documents and over time make changes to or renew their policy;
- Potentially, consolidation of the Andriod and IOS applications into a single solution powered by React Native, which performs the functionality of the current native applications but which also has web views

loading the aforementioned My Account functionality.

At the time of writing, full requirements are still being refined and can be seen at here.

Languages and Frameworks

As the aim of Digitisation is to deliver new greenfield web journeys and applications at speed there is a need to step back and assess language and framework choice particular for the front end applications.

Although the bulk of the existing front end web estate across Green Flag and indeed the Group as a whole is in Angular, there is Group level appetite to attempt to pivot to React for web applications and React Native for mobile applications where and when this makes sense. Certainly, a pivot towards React could achieve the following benefits:

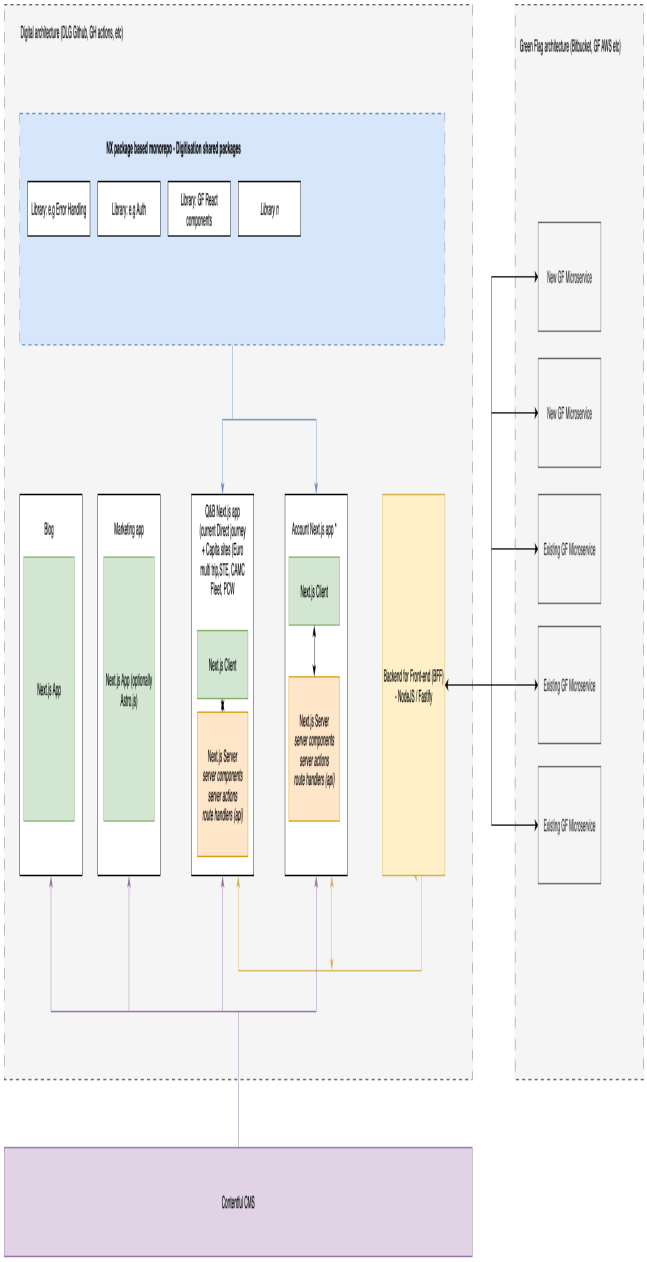
- Increased developer pool with which to resource the development team
- Quicker time to market
- Increased learning opportunities and opportunities for cross pollenation
- Potential for increased code sharing / engineer pooling between native and web applications

However, the key consideration to this is 'where and when this makes sense'. That is to say, it may make sense for certain aspects of Digitisation but not for others, depending on the unique requirements of each aspect. There is unlikely to be a 'one size fits all'. For

example, complex agent based internal systems may be best to remain Angular powered due to the sheer volume and complexity of forms, business based logic, and validation requirements where the ‘belt and braces’ approach of Angular shines.

The decision to proceed with React for the digitisation project is recorded in the following Key Decision Decision: KDD001 Front end framework

Front End Architecture Proposal - ideal state



* Existing native mobile applications (or new React Native app) to load My Account as webview. The Account Next.js app is depicted as one application for brevity but is likely to be comprised of separate sub applications, either through Module Federation or other techniques.

NX package based monorepo - front end libraries

A [package based NX monorepo]() for common functionality, aka library code will be created, to share common code and patterns between specific sub-applications. NX is a front end architecture pattern used existing in other areas of DLG. When coupled with semantic release, this allows shared library code to be

developed and new versions of packages published quickly and efficiently. Consuming applications can then decide when or if to update to the new version. This helps to ensure that each application does not have a specific implementation of a feature that is common between applications, ensuring consistency in solution as the project progresses.

The primary reason for not using an integrated NX monorepo containing the applications as well is largely due to AI considerations. With all digitisation applications in one repository, it is conceivable that codebase size grows at a larger rate than AI tokens, losing context and as a result AI being less ineffective. This does however mean that updates to library code will not be quite as timely to release and subsequent consume an updated version in the required application/applications.

Digitisation applications

Each of the sub applications will have the ability to be deployed separately, potentially using differing versions of claims-react-components or the library code packaged by the NX package based monorepo described above. However, it is recommended to keep the dependencies consisteny and up to date across the applications for security and compatibility reasons. As much code as possible should be shared between the applications via the library monorepo.

Marketing Site

The Marketing site is best placed to use either Next.js or Astro.js. Irrespective of option, the preferred method is server side rending (SSR) rather than static site generation at. build time (SSG).

Contentful web hooks would be required to setup to reflect content changes without any code releases. SSR is the preferred rendering option - client side rendering is discounted for the marketing site due to the risk of search engine spiders not being able to crawl a javascript app to it's full potential.

Option 1 - Astro.js

Astron.js is a well regarded framework for delivering performant, content driven websites. It's default approach is to deliver content pre-rendered at build time (SSG) rather than SSR or client side rendering. However, you can choose to render some or all of your routes on demand by a server when a route is requested.

Astro aims to improves website performance, sending lightweight HTML to the browser with zero unnecessary JavaScript overhead. It supports an 'islands architecture' in order to bring these performance optimisations and is well regarded in terms of performance around Core Web Vitals, and SEO. Astro has build in file structure based routing and typescript support out of the box.

As Astro supports every major UI framework (including React), framework components can theoretically be shared between the marketing

site and the other applications, resulting in less duplication and *potentially* sharing of common UI components with the other applications. However as the requirements for this site is understood to be largely static (except for Contentful CMS integration), vanilla Astro components could well suffice without needing a React integration. If client side hydration can be avoided as much as possible, this should bring the highest performance in the browser.

Sharing a component library like Mui between an Astro.js based marketing site and the My Account / Quote and Buy applications is possible. However, as Mui is a React based component library - to use this, React components rather than plain vanilla Astro components might be required. These React components should be rendered server side as and where possible rather than client side for optimum performance, to avoid the React library being loaded client side.

However, as the UI components needed for the marketing site should be limited in number (e.g, card, header, image block, accordion, button, link) using a lightweight CSS library such as Tailwind may be an acceptable choice to use in conjunction with vanilla Astro components for a highly performant experience.

Astro supports components across a number of different frameworks so is more flexible, and is anticipated to have a low barrier of entry in terms of understanding.

Option 2 - Next,js (already being developed by AI team)

Next.js is a powerful React framework that simplifies web development by providing pre-configured tooling, features, and optimizations for building fast, interactive applications. It's built on top of React and offers advantages like server-side rendering, static site generation, and built-in routing

Although Astro currently has the edge in performance, the benefits of implementing Next.js is that the estate is using the same framework therefore sharing components particularly those backed by a component library (whether that be Chakra, Mui or another) is easier. The trade off in this regard would be simplicity and potentially performance considerations. Note that Blog development has already commenced using the Chakra component library. This should ideally be standardised so the whole Digitisation estate is using MUI for consistency and development speed.

Discounted options

- Remix is a viable option but doesn't support SSG by default, just SSR. Potentially larger barrier to entry since Next.js is the most popular React framework
- Gatsby has low developer satisfaction according to the State of Javascript 2024 survey.

My Account / Quote and Buy

The intention is that Next.js is the best suited React library/framework for the 'My Account' and 'Quote and Buy' applications for the reasons outlined on the [KDD002 React framework decision](#).

The My Account application is estimated to contain a vast array of features (renewals, quotes, amendements, documents etc). For brevity in the above diagram it is depicted as one application, although it is likely that this is broken down into constituent parts (via Module Federation or alternative means) to make this more manageable.

The proposal assumes the applications are Dockerised applications rather than statically hosted via (for example) S3, since they are essentially full stack applications requiring a server, and can be hosted on ECS or similar.

Backend for Frontend (BFF)

The BFF is also known as a façade layer that sits between the frontend app and backend microservices. It takes care of orchestration i.e. where several BE services needed to fulfil single FE request. The API design of the BFF is driven by needs of the frontend app (consumer driven) and is usually owned by the same team or a sub-team.

Some of the advantages of the BFF are as follows:

- Existing pattern that is already used at Green Flag
- Simplify dependency management for FE UI team, single dependency on the BFF team allowing them to focus on end-user needs
- Low direct coupling to BE services, with BFF microservice acting as a ‘firebreak’ for any changes
- Can use a ‘consumer driven’ API to meet the specific needs of the FE code
- Avoid ‘polluting’ backend services with additional correlation IDs or data required for orchestration – we can deal with this at the BFF layer
- Separation of concerns for UI and orchestration logic making each faster to build, test and maintain
- Can deploy UI for testing with a single supporting mock/stub/simulator
- Can test BFF microservice simply at API level, no UI interaction needed
- Observability and monitoring simplified, can quickly isolate issues – a single frontend action corresponds to a single trace to the BFF microservice
- Simplified session management (if needed) for FE code, maintain session with BFF microservice and not with multiple BE components
- A common place to put authorisation

Next.js has the concept of server components and therefore can take on the function of a BFF. However, doing so comes with several disadvantages. The main one is high coupling

of BFF with Next.js - cannot reuse one BFF between separate, similar Next.js applications.

BFF patterns can be 1:1 or in some cases 1:many.

The proposal is for a single Node.js based BFF supporting all digitisation web frontends. Node.js has been chosen in order for front end teams to be able to effectively own the UI and BFF.

The reasons for omitting mobile applications from this BFF for now is:

- **Improved performance:** reducing unnecessary data transfer and processing to clients that may not need it
- **Enhanced scalability:** Individual BFFs can scale independently, allowing for better resource allocation and handling of traffic spikes.
- **Increased autonomy for frontend teams:** Frontend teams can develop and deploy their BFFs without impacting other teams, leading to faster iteration cycles.
- **Improved fault isolation:** Issues in the BFF are less likely to affect other parts of the system.

If there are modifications that we're considering making in the BFF for web front ends which would actually be good generic modifications to individual microservices so that other areas (e.g. mobile) could take advantage of them, then we should move logic down to microservice level.

In terms of BFF framework, there is no need to reinvent the wheel, with the Partnerships

tranche investing in Fastify.js for a number of reasons. One of the key requirements for that project was to run serverless via API Gateway and a single “fat” AWS Lambda, with Fastify supporting that requirement out of the box.

Green Flag vs Direct Line Group infrastructure

The proposal is for the digital front ends (including the libraries) and the BFF to be hosted on DLG Github organisation as is the expected group level standard.

However, the BFF will need to call some pre-existing Green Flag microservices and some new ones. Until there is a migration plan to move them (perhaps within Tranche 5), there is the assumption that these (existing and new) remain on Green Flag architecture.

DLG Github gives easier access to consuming existing DLG packages like the claims react components and data layer libraries if required. These digital front ends and associated libraries will be able to leverage the likes of Github Actions, SonarQube by being hosted on DLG Github.

CMS Integration

Contentful has been previously chosen as the Group headless CMS platform of choice.

Contentful’s usefulness will be most apparent for the Marketing site where non developers can tweak wording, content and more via the Contentful tool.

However, certain areas of the My Account / Quote and Buy site should be set up to be

configurable. It is however important to note that for these areas, there will be limitations as to what content can be changed. For example, re-ordering pre-existing questions that currently reside on the same page could be possible. However moving a question from the first page of the user journey to the last will not be supported. This is because of the touchpoints and contracts between the UI and backend microservices; fundamental changes like these that may alter what data is exchanged at a certain point in the journey would not be permissable without engineering effort. The recommendation is for Q&B contenful integration to be as light as possible and focused primarily on items of truly static content like header/footer, and flash messaging (outage notifications etc).

When content needs to change, the content authors should be able to make the changes in a safe environment, be able to preview them on the site in a non-production environment and there must be a peer review and approval process (i.e. CR) before publishing.

Contenful can be used on Astro via the Contentful SDK and also with Next.js

Contenful integration is not proposed to go via the BFF but instead directly, since it has strong CDN capabilities that would be lost or have to be reimplemented in the BFF. The Contenful content is not deemed to be sensitive.

Testing approach

At a very high level, various levels of automated testing should exist.

The testing approach is described in more detail on the Digitisation - UI and BFF Testing Strategy page.

Observability (FE)

Other areas of Group have published a comprehensive proposal on observability for front end applications where Application Performance Monitoring & Error Tracking Software was the proposed tool of choice, closely followed by New Relic. Whilst there are no challenges on the functional differences between Sentry and New Relic, the latter is already well embedded into the Green Flag ecosystem.

New Relic is current used to gain valuable insights across the whole estate, UI and microservice combined. Having this single view of application performance is extremely helpful in observability as correlations can help to diagnose issues quicker (certainly quicker than having to log into multiple tools, as would be the case if implementing Sentry). The power of synthetic monitors in catching issues before the arise is extremely important rather than relying on session replays of issues after they have occurred.

The New Relic Green Flag contract is pre-existing, so there is no increased cost in deciding to keep using New Relic. New Relic could not simply be switched off without losing observability on various other areas of

Green Flag (Staff Web for example). Additionally the Sentry plan would need modifying to Enterprise. Therefore, using New Relic seems the most sensible from a cost, disruption and risk perspective.

Observability (BFF)

Cloudwatch would be used for logs, metrics and traces.

Static code analysis

The Green Flag Bitbucket instance already has Codacy support, but the wider group use SonarCube for static code analysis.

Therefore the expectation is that SonarQube will be adopted for the digital front ends, associated library code, BFF and any new microservices, and shall run checks on every merge to main or release branches that trigger deployments. Merging will be blocked if these checks fail.

For changes to any pre-existing microservices hosted on Green Hub bitbucket, Codacy static code analysis should be already implemented.

Implementation is detailed on the Digitisation - UI and BFF Static Code Analysis page.

CSS Frameworks / Component Library

CSS framework / component library options can be seen in KDD003 Component / CSS library.

It is likely that either MUI (integration) or Chakra will be implemented via a Green Flag UI component library housed in the monorepo, where the UI components can be

shared across Digitisation applications. Tailwind and the use of Motor Claims Hub internal component library (built on top of Mui) have been largely discounted.

Tech Stack - specific tools

We will work in an agile way, including reflecting on approaches and adapting where appropriate. This is intended as a sensible starting point and not set in stone.

Frontend - marketing site

- React (Next.js latest version) or AstroJS (latest) with server side rendering and typescript
- CSS framework / component library (TBC)
- Contentful (latest)
- Node.js LTS (22.17.1 at the time of writing)
- Eslint (Linting)
- Prettier (Formatting)
- Vitest (Unit testing)
- Playwright (End to end testing)
- New Relic (Monitoring)
- Tealium TMS integration via a data layer library, which will be responsible for loading other marketing tools such as ContentSquare
- One Trust cookie management
- SonarQube static code analysis

Digital frontend: My Account, Quote and Buy

- React (Next.js latest version)
- CSS framework / component library (TBC)
- Contentful (latest)
- Node.js LTS (currently 22.16.0)
- Eslint (Linting)

- Prettier (Formatting)
- Vitest or Jest (Unit testing)
- Playwright (End to end testing)
- NewRelic (Monitoring)
- Tealium TMS integration via a data layer library, which will be responsible for loading other marketing tools such as ContentSquare
- One Trust cookie management
- Stripe integration via existing payment intent Lambda
- SonarQube static code analysis

Backend for Frontend

- Node.js LTS (currently 22.16.0)
- Fastify - see KDD007 - Backend For Frontend (BFF) Microservice for UI for a previous discussion of frameworks conducted by the Partnerships team - the main reason here is Fastify natively supports deployment to AWS Lambda, unlike the leading framework Express
- Typescript (latest)
- Eslint (Linting)
- Prettier (Formatting)
- Jest (Testing)
- CloudFront for logs, metrics, traces
- Codacy static code analysis